

Design & Implementation of Event-Driven Microservices Architecture using MiniStack

Abiyu Salam

Step 0. Export Environment Variables

```
export AWS_ACCESS_KEY_ID="mock"
export AWS_SECRET_ACCESS_KEY="mock"
export AWS_DEFAULT_REGION="us-east-1"
```

Step 1. Penyusunan Infrastruktur Dasar

```
# 1. Membuat Tabel DynamoDB 'Orders' dengan Stream Aktif (NEW_IMAGE)
aws --endpoint-url=http://localhost:4566 dynamodb create-table \
    --table-name Orders \
    --attribute-definitions AttributeName=orderId,AttributeType=S \
    --key-schema AttributeName=orderId,KeyType=HASH \
    --provisioned-throughput ReadCapacityUnits=5,WriteCapacityUnits=5 \
    --stream-specification StreamEnabled=true,StreamViewType=NEW_IMAGE

# 2. Membuat Antrean SQS 'order-processing-queue'
aws --endpoint-url=http://localhost:4566 sqs create-queue \
    --queue-name order-processing-queue

# 3. Verifikasi Identitas Email di SES MiniStack (Wajib Mode Sandbox)
aws --endpoint-url=http://localhost:4566 ses verify-email-identity --email-
address store@example.com

aws --endpoint-url=http://localhost:4566 ses verify-email-identity --email-
address ahya@example.com
```

Step 2. Setup Source Code & Deploy Lambda

```
# 1. Pastikan lu udah bikin file source code berikut di direktori lu

# 2. Zip masing-masing file source code
zip create_order.zip create_order.py
zip stream_filter.zip stream_filter.py
zip worker.zip worker.py
```

```
zip get_order.zip get_order.py
zip notification.zip notification.py
```

3. Deploy ke Lambda MiniStack

```
aws --endpoint-url=http://localhost:4566 lambda create-function --function-
name CreateOrder --runtime python3.9 --role
arn:aws:iam::000000000000:role/lambda-role --handler
create_order.lambda_handler --zip-file fileb://create_order.zip
```

```
aws --endpoint-url=http://localhost:4566 lambda create-function --function-
name StreamFilter --runtime python3.9 --role
arn:aws:iam::000000000000:role/lambda-role --handler
stream_filter.lambda_handler --zip-file fileb://stream_filter.zip
```

```
aws --endpoint-url=http://localhost:4566 lambda create-function --function-
name Worker --runtime python3.9 --role arn:aws:iam::000000000000:role/lambda-
role --handler worker.lambda_handler --zip-file fileb://worker.zip --timeout
15
```

```
aws --endpoint-url=http://localhost:4566 lambda create-function --function-
name GetOrder --runtime python3.9 --role
arn:aws:iam::000000000000:role/lambda-role --handler get_order.lambda_handler
--zip-file fileb://get_order.zip
```

```
aws --endpoint-url=http://localhost:4566 lambda create-function --function-
name NotificationLambda --runtime python3.9 --role
arn:aws:iam::000000000000:role/lambda-role --handler
notification.lambda_handler --zip-file fileb://notification.zip --timeout 10
```

Step 3. Hubungkan Event Source Mapping (Pipa)

1. Dapatkan ARN Stream dari Tabel Orders terlebih dahulu

```
STREAM_ARN=$(aws --endpoint-url=http://localhost:4566 dynamodb describe-table
--table-name Orders --query "Table.LatestStreamArn" --output text)
```

2. Sambungkan DynamoDB Stream ke Lambda StreamFilter (Gunakan TRIM_HORIZON!)

```
aws --endpoint-url=http://localhost:4566 lambda create-event-source-mapping \
    --event-source-arn $STREAM_ARN \
    --function-name StreamFilter \
    --starting-position TRIM_HORIZON \
```

```

--batch-size 10

# 3. Smbungkan SQS Queue ke Lambda Worker
aws --endpoint-url=http://localhost:4566 lambda create-event-source-mapping \
    --event-source-arn arn:aws:sqs:us-east-1:000000000000:order-processing-queue \
    --function-name Worker \
    --batch-size 1

# 4. Smbungkan DynamoDB Stream ke Lambda NotificationLambda (Pemicu SES)
aws --endpoint-url=http://localhost:4566 lambda create-event-source-mapping \
    --event-source-arn $STREAM_ARN \
    --function-name NotificationLambda \
    --starting-position TRIM_HORIZON \
    --batch-size 10

# 5. CRITICAL STEP: Berikan jeda waktu agar MiniStack menstabilkan State Pipa
menjadi 'Enabled'

sleep 5

```

Step 4. Uji Pembuatan Transaksi Baru (Injeksi)

```

aws --endpoint-url=http://localhost:4566 lambda invoke \
    --function-name CreateOrder \
    --payload '{"customer": "Ahya", "product": "Mechanical Keyboard",
"quantity": 2}' \
    response.json && cat response.json

```

Step 5. Verifikasi Perubahan Status (COMPLETED)

```

# Notes : Gantilah string ORDER_ID dengan nilai orderId asli yang keluar dari
output Step 4!

aws --endpoint-url=http://localhost:4566 lambda invoke \
    --function-name GetOrder \
    --payload '{"orderId": "ORDER_ID"}' \
    check.json && cat check.json

```

Step 6. Verifikasi Eksekusi Notifikasi SES

Intip log NotificationLambda untuk memastikan email terkirim asinkronous lewat SES lokal

```
aws --endpoint-url=http://localhost:4566 logs filter-log-events \  
    --log-group-name /aws/lambda/NotificationLambda \  
    --max-items 20
```